

- 異なる画面間のナビゲーションを簡単に整理できる.
- MaterialApp の主なメリット:
 - ◇ すぐに使える構造 → 開発時間を短縮.
 - ◇ Material Design 準拠 → UI の一貫性を確保.
 - ◇ 簡単なナビゲーション管理 → 複数画面の管理が容易.
- d. 比較: Java におけるアプリのセットアップ方法
 - Java (Android 開発) では,UI コンポーネントは通常 XML ファイルで作成される.
 - Flutter では,MaterialApp とウィジェットを使って UI を宣言的に構築する.
 - Flutter のアプローチは,コードの複雑さを減らし,保守性を向上させる.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

- 主な違い:

機能	Java (Android)	Flutter (Dart)
UI の構造	XML ファイルでレイアウトを定義	Dart コード内で直接 UI を作成
ナビゲーション設定	Intent や Fragment を使用	MaterialApp で簡単に管理
複雑さ	コードが長くなりやすく, 手動更新が必要	シンプルで宣言的な UI モデル

ステップ 9 : Flutter の Scaffold とは？

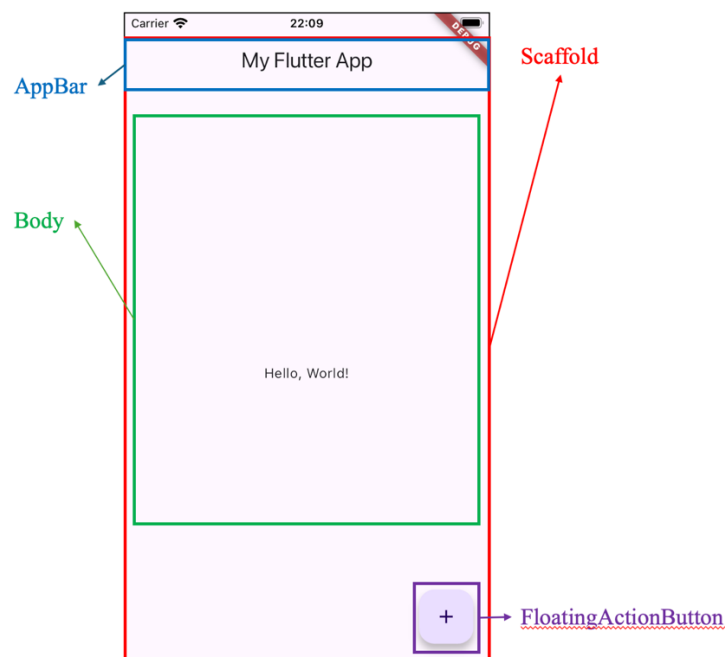
- a. Scaffold とは？
 - Scaffold は,Flutter アプリの画面の基本構造 (スケルトン) を提供する.
 - AppBar,body,FloatingActionButton などの主要な UI 要素を整理する.
 - UI レイアウトを簡素化し,アプリのインターフェースを構築しやすくする.
 - Scaffold の主要な UI コンポーネント:
 - ◇ AppBar → タイトルやナビゲーションボタンを表示するトップバー.
 - ◇ body → 画面のメインコンテンツ領域.
 - ◇ floatingActionButton → すぐにアクションを実行できるフローティングボタン.
- b. Scaffold の基本的な例
 - Flutter のコード例:

```
Scaffold(
  appBar: AppBar(
```

```

    title: Text('My Flutter App'),
  ),
  body: Center(
    child: Text('Hello, World!'),
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {
      print('Button Pressed!');
    },
    child: Icon(Icons.add),
  ),
);

```



■ Scaffold の主要な構成要素

◇ appBar - 上部のナビゲーションバー

- ナビゲーションとタイトルを表示するトップバーを作成する.
- タイトル (Text) やアクションボタンを含むことが多い.
- appBar プロパティ内で AppBar ウィジェットを使用する.

◇ body - メインコンテンツエリア

- 画面の中で最も大きな部分で,ほとんどのウィジェット (テキスト,画像,ボタンなど) を配置する.
- Center, Column, Row などのウィジェットを使ってレイアウトを整理する必要がある.
- Text, ListView, GridView などのウィジェットを含むことが多い.

◇ floatingActionButton - クイックアクションボタン

- コンテンツの上に「浮く」ボタンで,クイックアクションに使用される.
- アイコンや他のウィジェットを内部に配置する必要がある.

- アイテムの追加やコンテンツのリフレッシュなどのアクションに使われる。
- c. なぜ Scaffold が重要なのか？
- 構造化されたレイアウトを提供し,画面を作成しやすくする.
 - アプリの異なる部分で一貫した UI 体験を実現する.
 - MaterialApp と統合し,スムーズなナビゲーションとテーマ管理を可能にする.
 - Scaffold のメリット:
 - ◇ 適切な構造を提供 → UI デザインを簡素化する.
 - ◇ ユーザー体験を向上 → モダンでレスポンスなレイアウトを実現.
 - ◇ Material Design と統合 → UI の一貫性を確保する.

まとめ

- Flutter アプリは main() と runApp() から始まり,MyApp ウィジェットを読み込む.
- MyApp は StatelessWidget を継承し,ルートウィジェットである MaterialApp を返す.
- MaterialApp の中で Scaffold が使用され,画面レイアウトが構築される.
- build() メソッドはウィジェットツリーを構築し,return によって UI が定義される.
- AppBar,body,floatingActionButton などの主要なウィジェットは Scaffold の中に配置される.
- Flutter のウィジェットシステムは階層構造になっており,すべてがウィジェットの中に構築されている.